OTUS

Kotlin Backend Developer.Professional

otus.ru

REC

Проверить, идет ли запись

Меня хорошо видно && слышно?

Транспорт с Kafka

Правила вебинара

Активно
участвуем

Off-topic обсуждаем в
OTUS-Kotlin-2025-05

Задаем вопрос
в чат или голосом

Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения

Индивидуально

Время, необходимое
на активность

Пишем в чат

Говорим голосом

Документ

Ответьте себе или
задайте вопрос

О курсе

Модуль 1: Введение в Kotlin	Введение	Базовые элементы	Функц. элементы	ООП		
Модуль 2: Расширенные возможности Kotlin	DSL	Корутины	Sequence, Flow	Multi-platform	Interop	Gradle
Модуль 3: Подготовка к разработке	Выбор проекта	Маркетинг аналитика	Security	Архитек-тура	DevOps	
Модуль 4: Разработка API	OpenAPI	Модели и маперы	Testing	Monitoring		
Модуль 5: Фреймворки и асинхронный транспорт	Spring	Ktor	WebSocket	RabbitMQ	Kafka	Serverless
Модуль 6: Бизнес-логика проекта	Основы БЛ	Модуль БЛ	Валидация			
Модуль 7: Работа с базами данных	Работа с БД	Репози-торий	Конкурен-ция	Postgres	Cassandra	Gremlin
Модуль 8: Развитие бизнес-логики проекта	Машина состояний	Auth	Основы ML	ML в бэкенде	Обзор экосист.	
Модуль 9: Проектная работа	Защита					

Окатов Сергей
автор программы, руководитель курса

- кандидат физ.-мат. наук
- 15 лет научной работы в университетах РФ, Европы, США
- 10 лет комм. разработки в стартапах и корпорациях (Сбер, Ланит) в качестве руководителя разработки, архитектора
- Специализация
  - Котлин
  - Большие данные
  - Распределенные системы
  - Архитектура

План полета

- О Kafka
- Работа с Kafka
- Подключение в проект
- Интеграция с логикой
- Рефлексия

Цели вебинара

После занятия вы:

1. Узнаете особенности Kafka
2. Сможете принимать решение о ее применении в проекте
3. Выполните интеграцию бизнес-логики в приложение
4. Узнаете шаблон разработки “Стратегия”

Смысл

Зачем вам это уметь

1. Kafka используется в качестве транспорта в микросервисной архитектуре, а также в качестве ESB
2. На Kafka строится много проектов
3. Kafka имеет собственные преимущества, из-за которых применяется в проектах

О Kafka

О Kafka

Общие сведения:

- Брокер сообщений
- Назначение – доставка сообщений от продюсера к консьмеру
- Один из инструментов BigData, справляется со сверхвысокими нагрузками
- Распределённая, горизонтально масштабируемая система
- Разработан LinkedIn
- Открыт в 2011
- Выпущен из Apache-инкубатора в 2012
- Современный конкурент - NATS

О Kafka

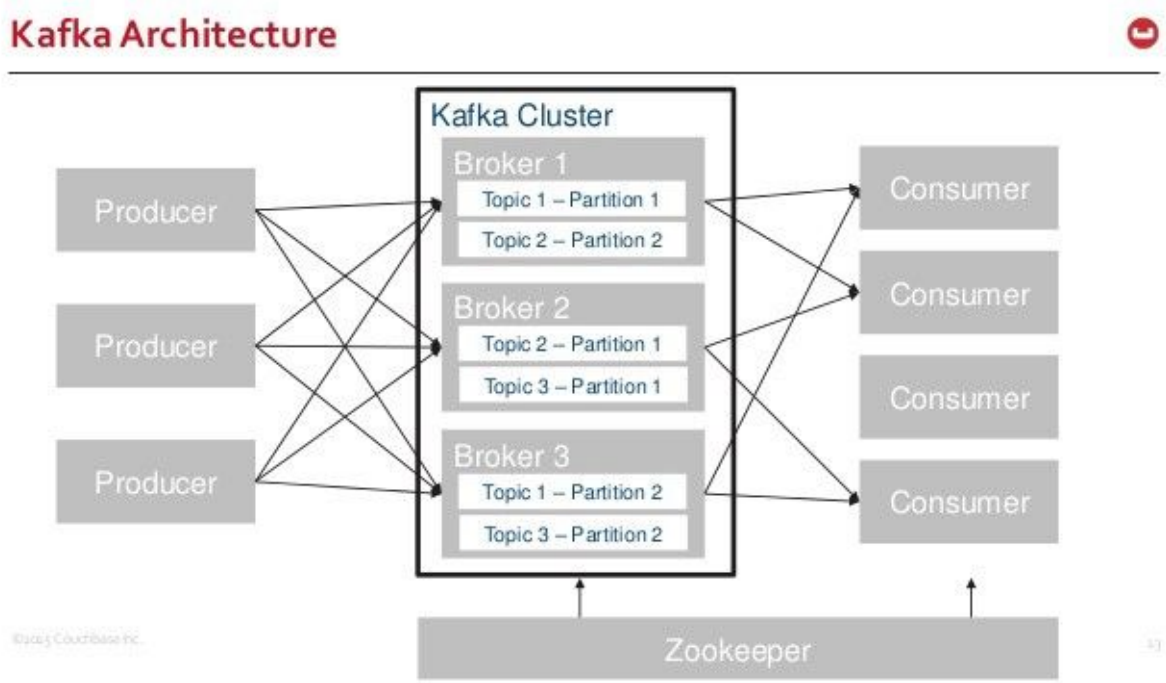
Kafka – это система, реализующая распределенный реплицируемый лог сообщений

- Распределенный – лог (топик) не целиком хранится на одной машине, а разбит на несколько секций (партиций), которые лежат на разных машинах
- Реплицируемый – логи хранятся в нескольких копиях, на случай отказов оборудования
- Лог – это упорядоченная последовательность сообщений

Элементы Kafka

Термины:

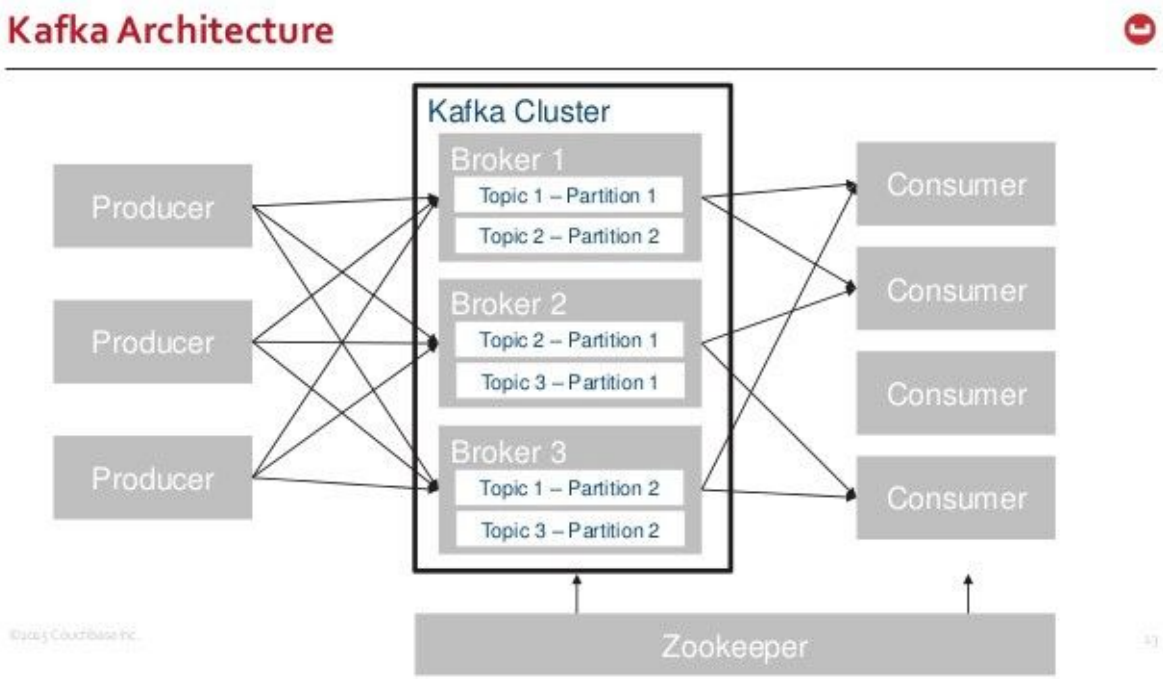
1. Producer – отправитель сообщений
2. Consumer – получатель сообщений
3. Topic – “очередь” сообщений
4. Partition – разделы топиков



Продюсер

Producer (client):

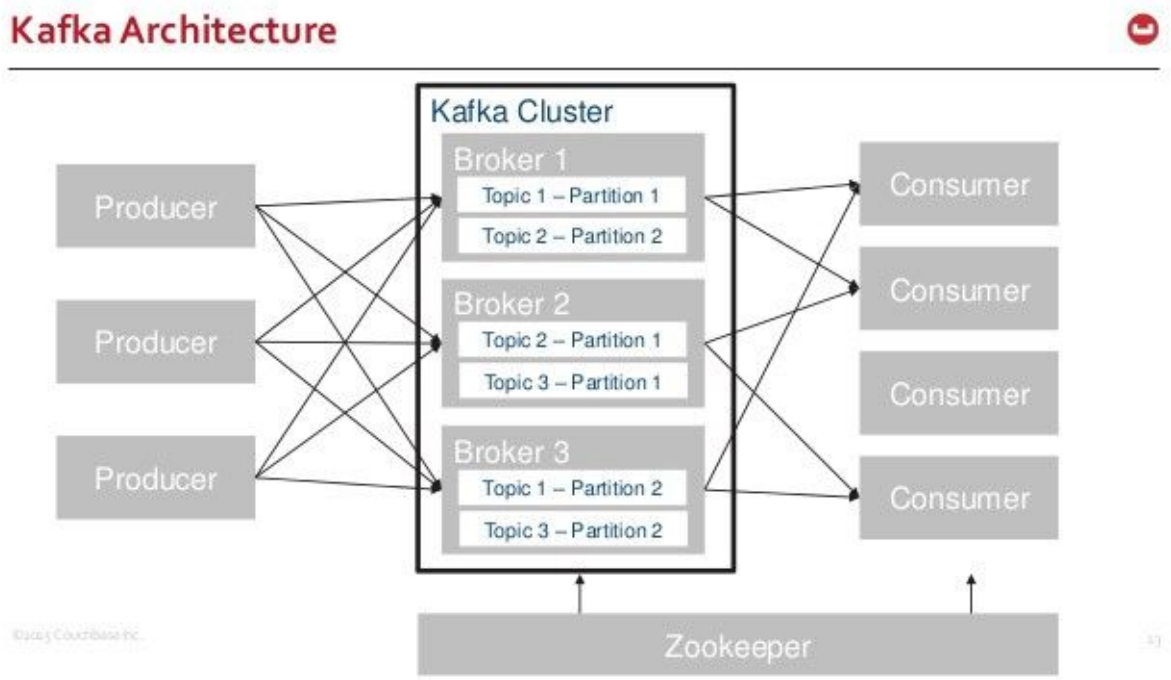
1. Пишет в партицию топика
2. Выбор партиции происходит на базе выбранного партишинера (interface Partitioner) или с явным указанием



Топик

Topic:

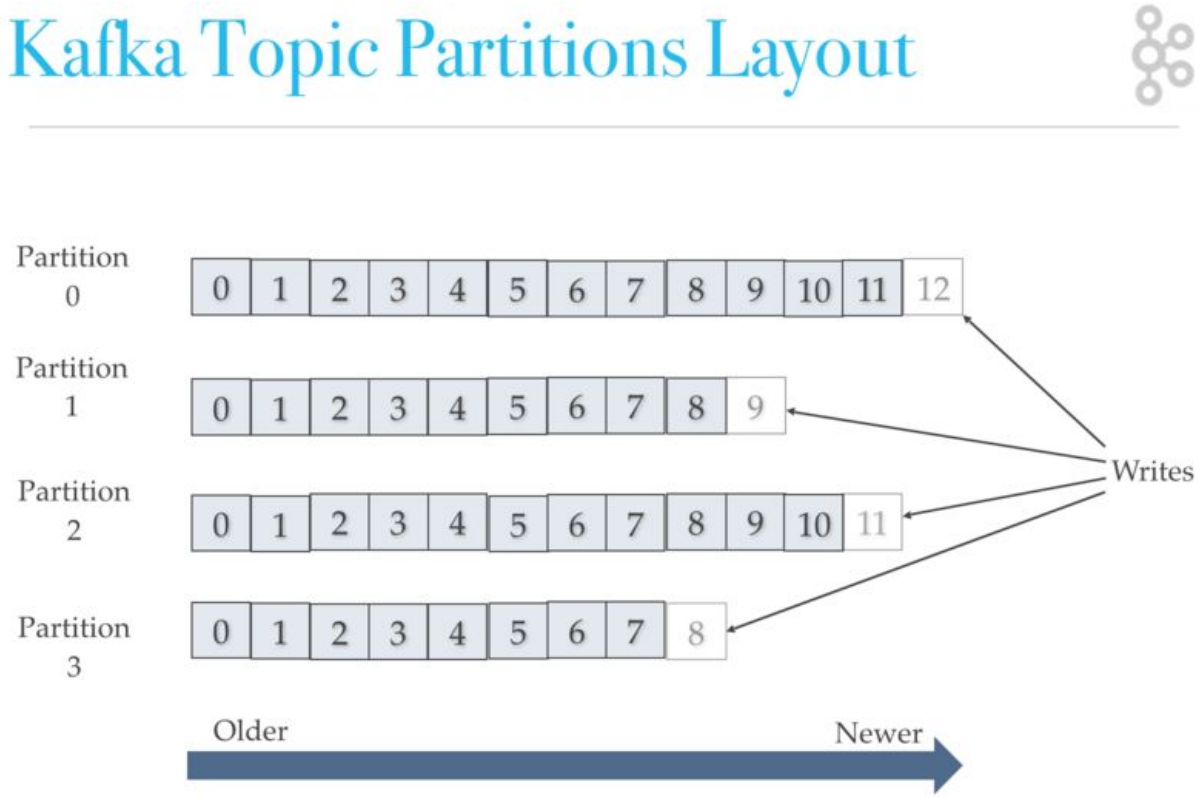
1. Логический “Канал” доставки сообщений, сообщения по одной теме
2. Разбит на одну и более партиций. Число фиксировано
3. Имеет TTL
4. Все сообщения доступны до момента удаления по TTL



Партиция

Partition:

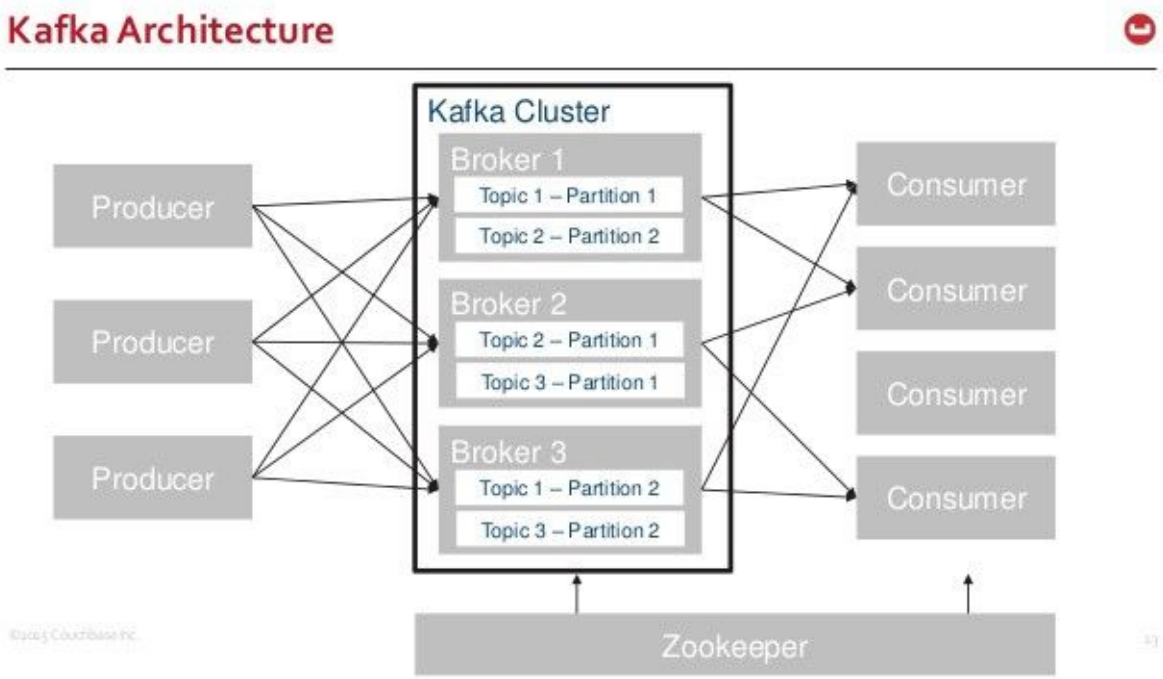
- Позволяет доставлять сообщения параллельно
- Обеспечивает масштабирование
- Может иметь реплики
- Резервируется в файловой системе
- Используется алгоритм последовательной записи



Консьюмер

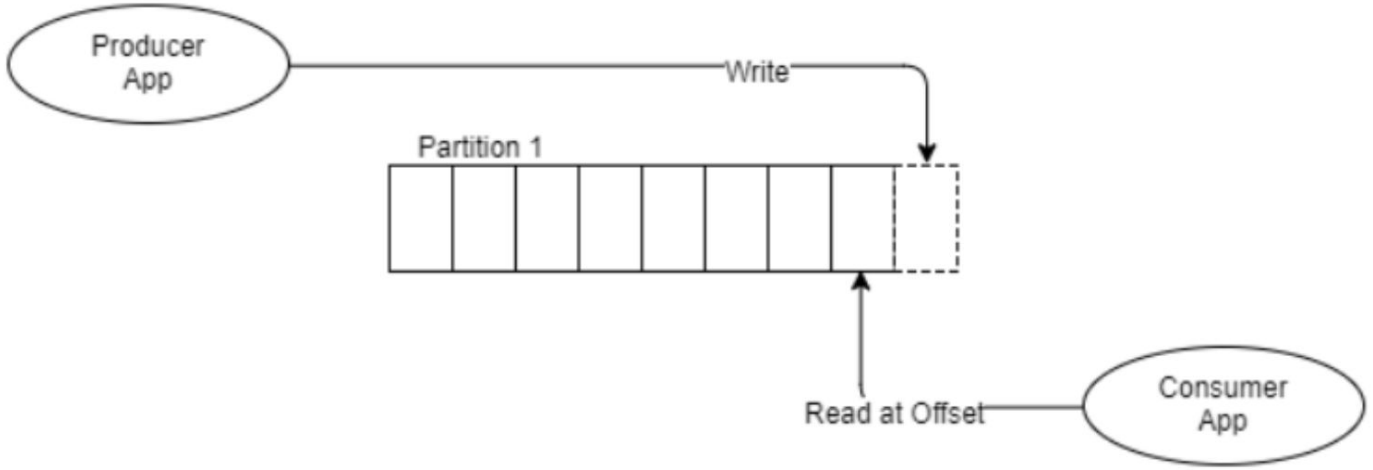
Consumer:

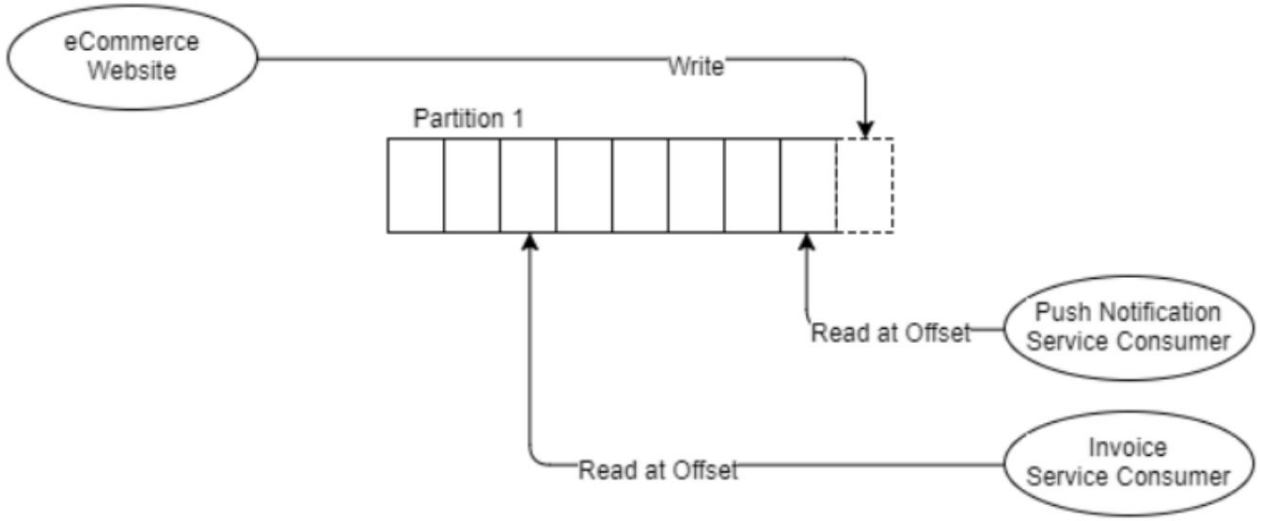
- извлекает сообщения из партиций
- может быть подключен к нескольким партициям
- группируются по GroupID
- Для конкуретного чтения топика 5 консьюмерами, нужно не менее 5 партиций с одинаковыми GroupID



Смещения

- Producer добавляет сообщение в распределенный лог сообщений
- Consumer читает из этого лога непрочитанные с прошлого раза записи
- Для каждой пары groupId-partition Kafka хранит у себя offset – указатель на сообщение, которое он прочитал последним.

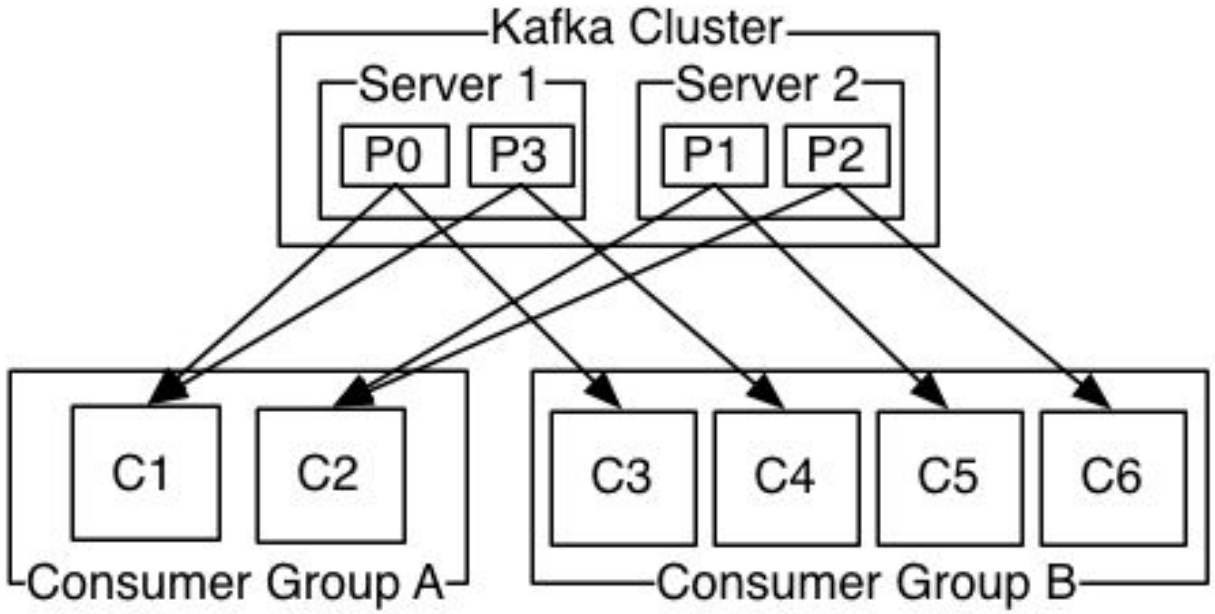




Группы консьюмеров

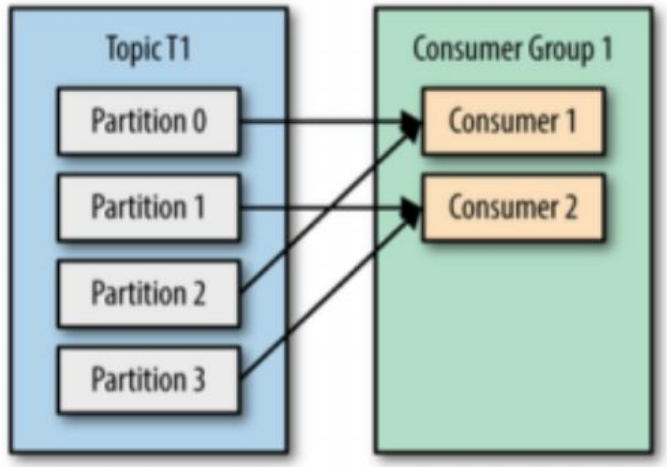
Consumer Groups:

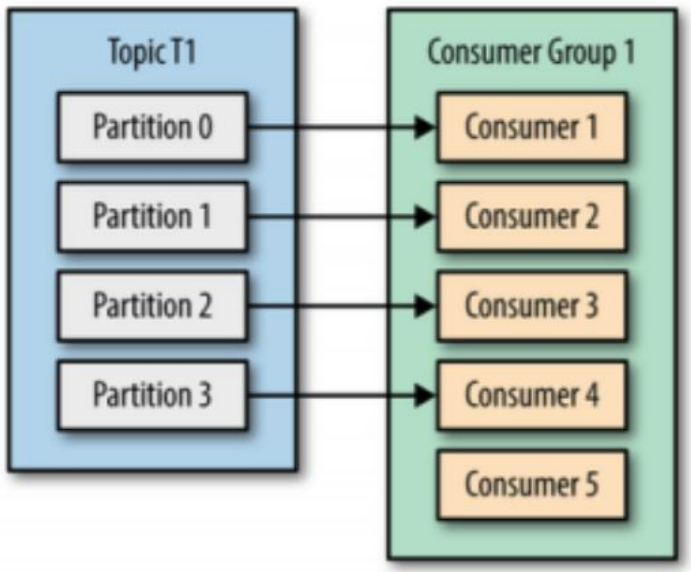
- Каждое сообщение получает один консьюмер из группы
- Разные группы получают каждое сообщение
- Консьюмер может читать из нескольких партиций
- Каждая партиция связана только с одним консьюмером из группы
- Для чтения топика 5 консьюмерами одной группы, нужно не менее 5 партиций
- Если консьюмеров больше, чем партиций – лишние не будут ничего получать

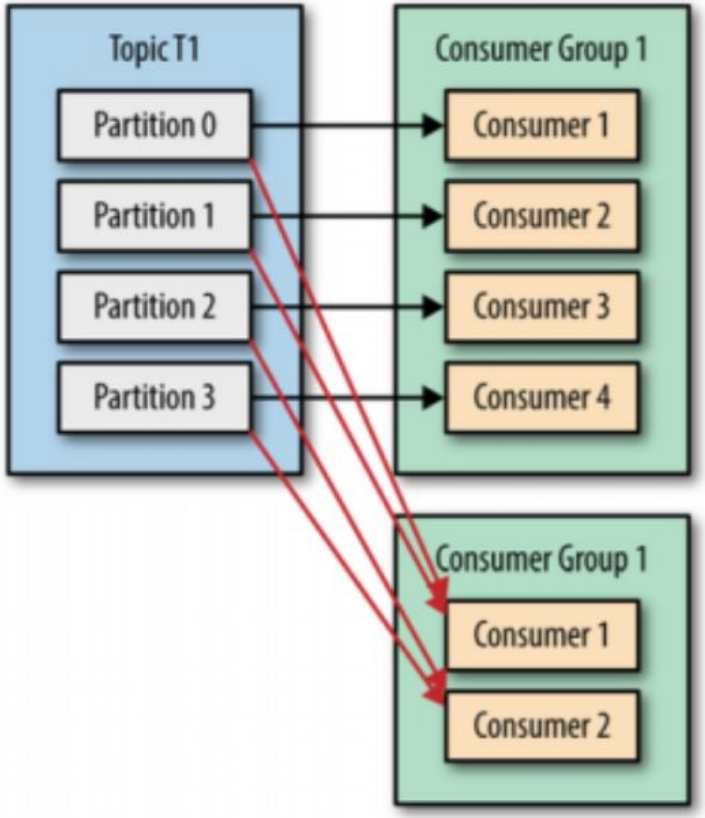


Связь консьюмеров и партиций

У каждой партиции должен быть только один конкурирующий консьюмер.
Один консьюмер может читать из нескольких партиций.

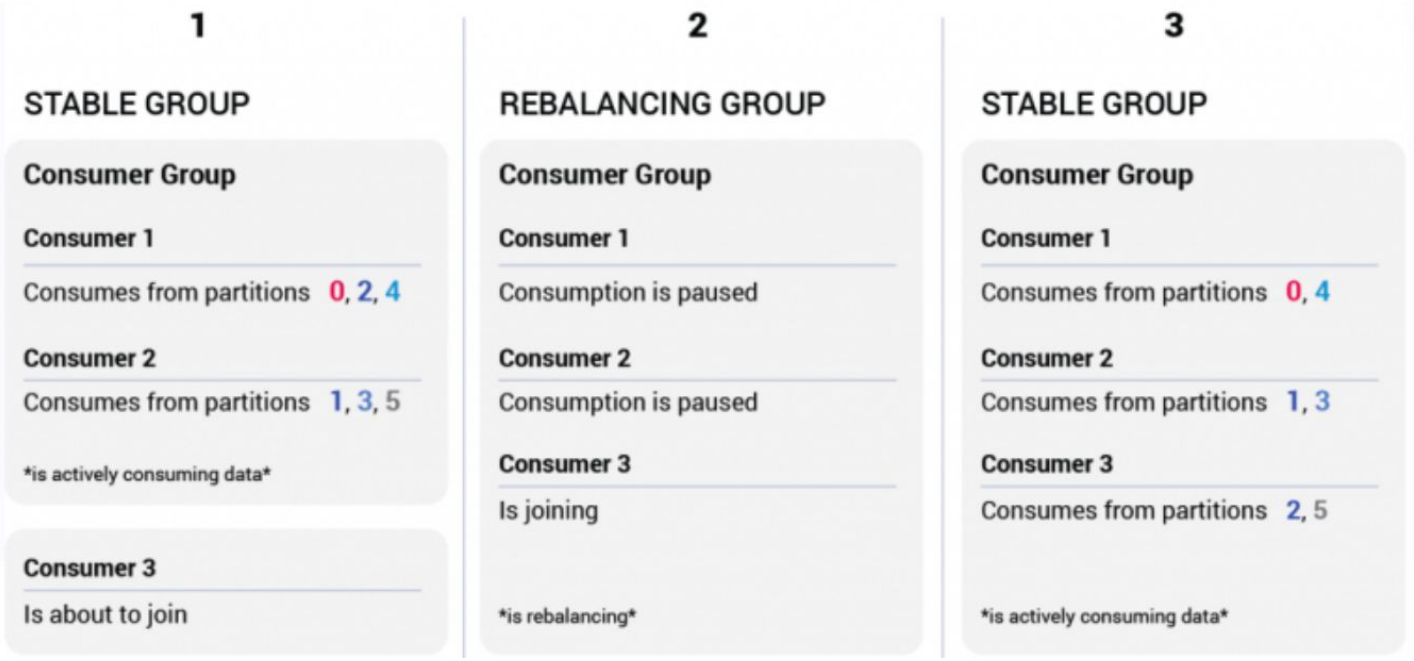






Перебалансировка кластера

Добавление или удаление Консьюмера в группу приводит к перебалансировке
При перебалансировке ни один из консьюмеров не читает сообщения из топиков.



Ключ сообщения

У сообщения в kafka может быть ключ – key.

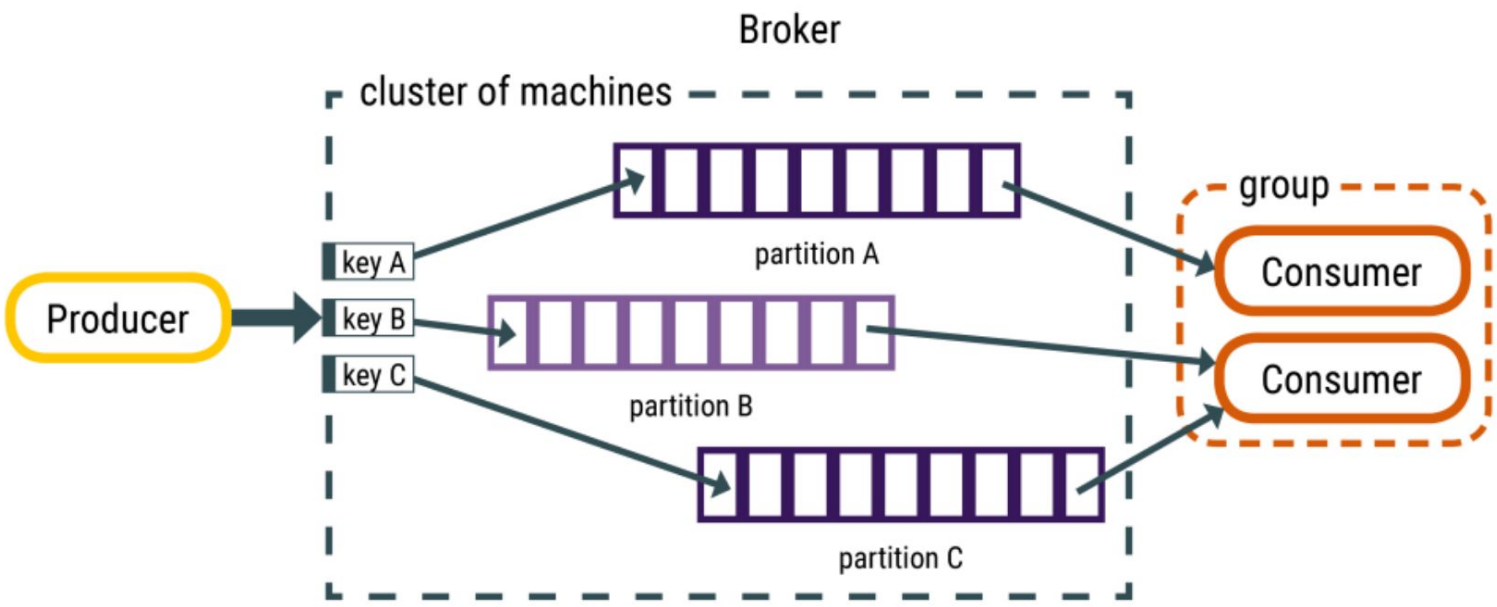
Типичное применение ключа – управление партицированием

Если key == null, распределение сообщений по партициям по RoundRobin

Если key != null, распределение сообщений по партициям по хэшу ключа

Log Compaction in

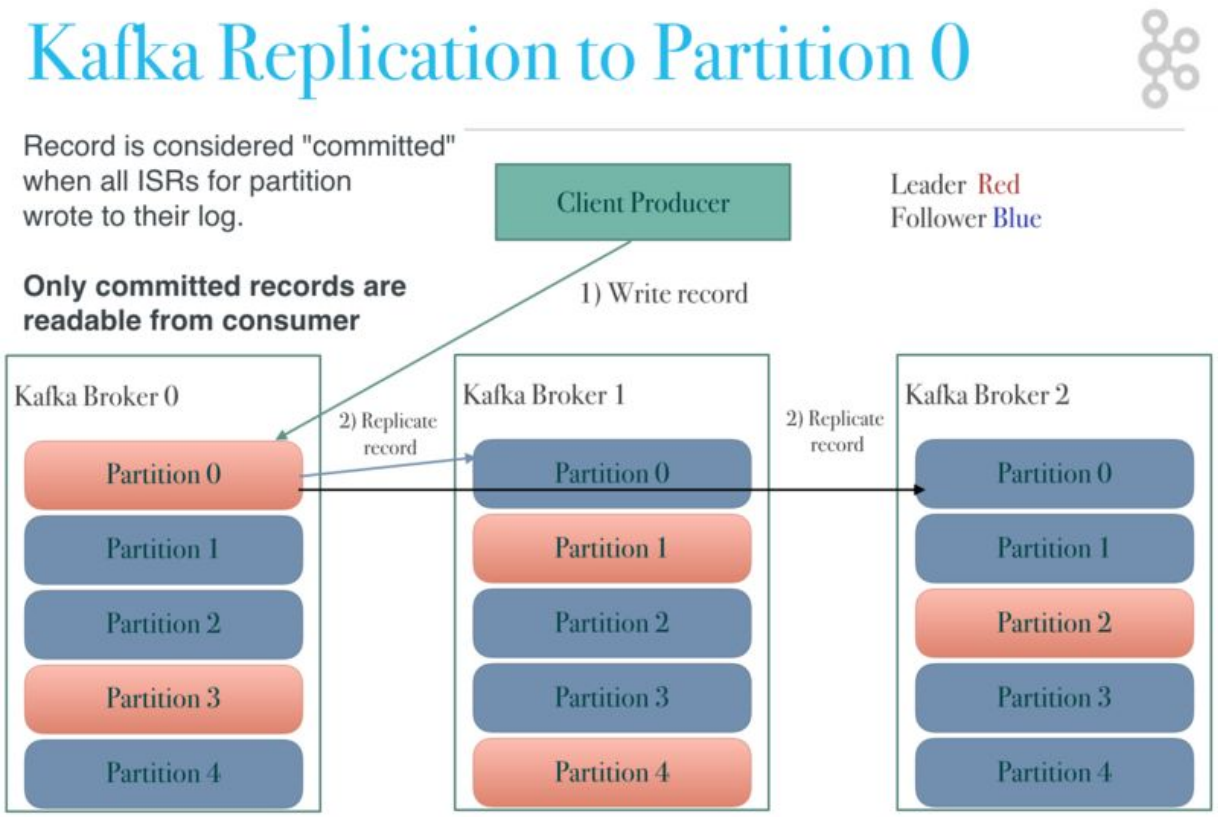
cleanup.policy = compacted



Репликация

Replication:

1. Обеспечивает гарантированную доставку сообщений в условиях сбоев
2. Партиция:
   a. Leader – основная
   b. In-Sync Replica – реплика, синхронизированная с лидером
3. Авторы Kafka советуют
   a. фактор репликации 1, если вы ок, с тем, что можете эти данные потерять,
   b. фактор репликации 3, если хотите, чтобы данные остались в сохранности.



Политики доставки Kafka

Семантики:

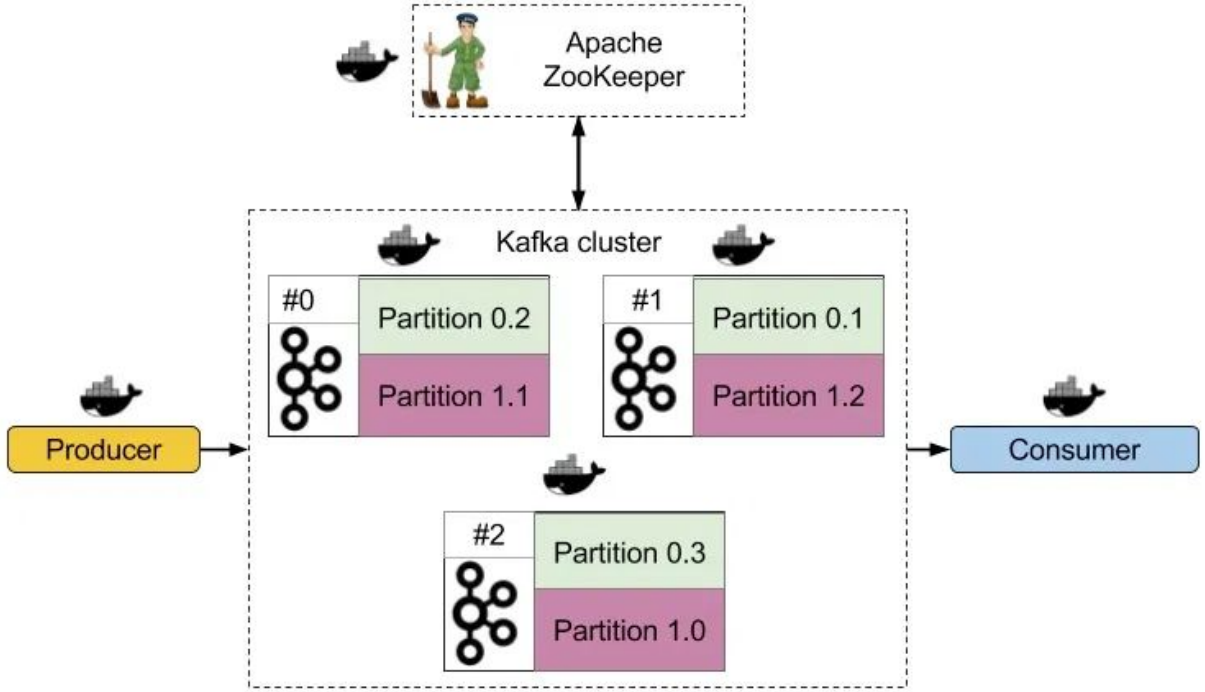
1. At least once:
   a. acks = all
   b. повторная отправка при таймауте/ошибке
2. At most once:
   a. без повторной отправки
3. Exactly once (20% медленнее):
   a. enable.idempotence=true (нумерация пакетов)
   b. транзакционная отправка
   c. processing.guarantee=exactly_once (streams api)
   d. потребитель:
      - read_committed
      - read_uncommitted

```
producer.initTransactions();
try {
 producer.beginTransaction();
 producer.send(record1);
 producer.send(record2);
 producer.commitTransaction();
} catch(ProducerFencedException e) {
 producer.close();
} catch(KafkaException e) {
 producer.abortTransaction();
  }
```

Кластер Kafka (legacy)

Кластеризация:

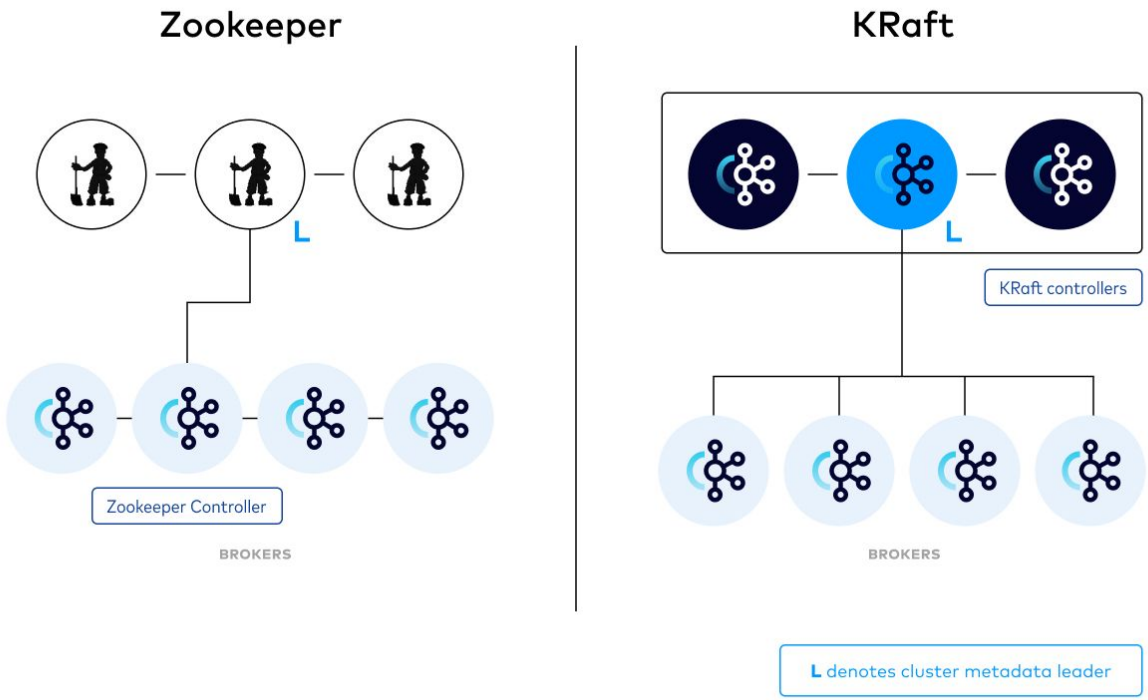
1. Распределенная архитектура. В качестве управляющего контроллера ZooKeeper
2. На разных узлах как партиции, так и реплики партиций
3. Клиент может подключаться
   a. к любому узлу
   b. к нескольким узлам
   c. через zookeeper
4. Клиент знает всю топологию кластера
5. Политики доставки …



Кластер Kafka Kraft (new)

Кластеризация:

1. Узел может быть broker, controller или два в одном
2. Более высокая масштабируемость и выше скорость на больших кластерах
3. Контроллеры участвуют в голосовании



Работа с Kafka

Библиотеки

1. kafka-clients – официальная библиотека
2. spring-kafka – Spring Boot Kafka
3. Ktor-kafka - https://github.com/gAmUssA/ktor-kafka

Интеграции

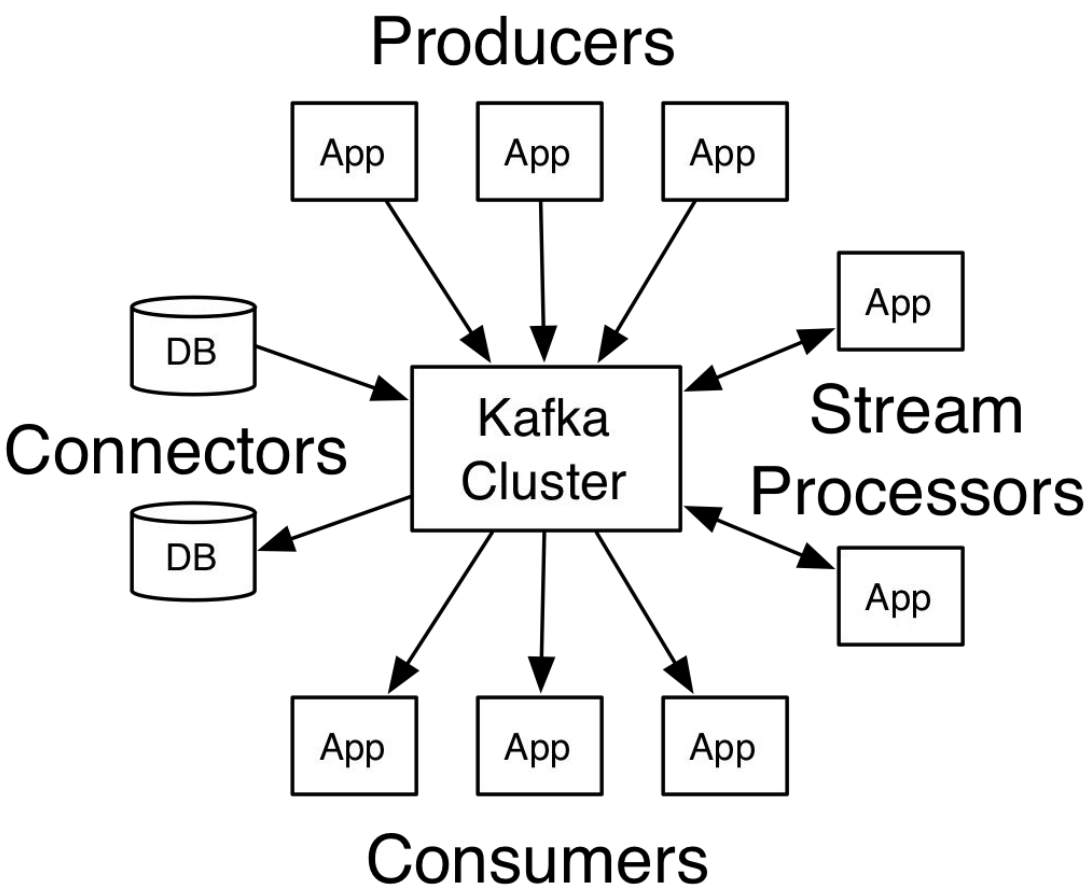
Интеграции с Kafka:

1. **Kafka-Connect** - интеграция с базами данных, другими источниками и потребителями данных (Confluent)
2. **Kafka-Streams** - Работа с потоками сообщений (объединение, фильтрация, агрегация)
3. **Camel-Kafka** - более эффективная замена Kafka-Connect

Интеграции

APIs:

1. Producer API – для публикации сообщений
2. Consumer API – для получения сообщений
3. Connector API – для соединения различных потребителей или продьюсеров (AMQP, SQL, REST, etc) к топикам Kafka
4. Streams API – для работы с потоками
5. Admin API – для управления кластером Kafka



Запуск Kafka в Docker

```yaml
version: "3.3"
services:
 zookeeper:
   image: wurstmeister/zookeeper:3.4.6
   expose:
     - "2181"
 kafka:
   image: wurstmeister/kafka:2.11-2.0.0
   ports:
     - "9094:9094"
   environment:
     # Во внутренней сети docker-compose слушаем адрес kafka:9092
     # Из внешней сети docker-compose будем слушать localhost:9094
     # Как альтерантива, можно вычислять адрес как-то так:
     # HOSTNAME_COMMAND: curl http://169.254.169.254/latest/meta-data/public-hostname
     # KAFKA_ADVERTISED_LISTENERS: INSIDE://:9092,OUTSIDE://_{HOSTNAME_COMMAND}:9094
     KAFKA_ADVERTISED_LISTENERS : PLAINTEXT://kafka:9092,OUTSIDE://localhost:9094
     KAFKA_LISTENERS : PLAINTEXT://0.0.0.0:9092,OUTSIDE://0.0.0.0:9094
     KAFKA_LISTENER_SECURITY_PROTOCOL_MAP : PLAINTEXT:PLAINTEXT,OUTSIDE:PLAINTEXT
     KAFKA_ZOOKEEPER_CONNECT : zookeeper:2181
     KAFKA_INTER_BROKER_LISTENER_NAME : PLAINTEXT
```

docker-compose -f docker-compose-kafka.yml up

Библиотеки: Original

Kafka:

- Создать consumer
- Создать producer
- Создать ProducerRecord
- Отправить

Нет потокобезопасности!

Каждый консьюмер в отдельном потоке

```
var producerRecord :ProducerRecord<String, String> = ProducerRecord(topic, message)

val map = mutableMapOf<String, String>()
map["key.serializer"]   = "org.apache.kafka.common.serialization.StringSerializer"
map["value.serializer"] = "org.apache.kafka.common.serialization.StringSerializer"
map["bootstrap.servers"] = "localhost:9092"

var producer = KafkaProducer<String, String>(map as Map<String, Any>?)
var future:Future<RecordMetadata> = producer?.send(producerRecord)!!
```

```
val props = Properties().apply {
    put("bootstrap.servers", brokers)
    put("group.id", "person-processor")
    put("key.deserializer", StringDeserializer::class.java)
    put("value.deserializer", StringDeserializer::class.java)
}
val consumer = KafkaConsumer<String, String>(props)

while(true) {
    val records = consumer.poll(Duration.ofSeconds(1))

}
```

Библиотеки: Spring

Spring:

- Создать приложение
- Создать очереди и обменники
- Создать слушателя

```
@Autowired
private KafkaTemplate<String, String> kafkaTemplate;

@PostMapping
public void sendOrder(String msgId, String msg){
    kafkaTemplate.send("msg", msgId, msg);
}
```

```
@EnableKafka
@SpringBootApplication
public class SimpleKafkaExampleApplication {

    @KafkaListener(topics="msg")
    public void msgListener(String msg){
        System.out.println(msg);
    }

    public static void main(String[] args) {
        SpringApplication.run(SimpleKafkaExampleApplication.class, args);
    }
}
```

Тестирование приложения со Spring Boot

Spring Boot:

- Подключить через аннотацию Embedded Kafka
- Получить бины консьюмера и продьюсера
- выполнить тест

```
@SpringBootTest
@DirtiesContext
@EmbeddedKafka(partitions = 1, brokerProperties = {
"listeners=PLAINTEXT://localhost:9092", "port=9092" })
class EmbeddedKafkaIntegrationTest {

    @Autowired
    private KafkaConsumer consumer;

    @Autowired
    private KafkaProducer producer;

    @Value("${test.topic}")
    private String topic;

    @Test
    public void givenEmbeddedKafkaBroker_whenSendingtoSimpleProducer()
      throws Exception {
        producer.send(topic, "Sending with own simple KafkaProducer");
        consumer.getLatch().await(10000, TimeUnit.MILLISECONDS);

        assertThat(consumer.getLatch().getCount(), equalTo(0L));
        assertThat(consumer.getPayload(), containsString("embedded-test-topic"));
    }
}
```

LIVE

Рефлексия

Вопросы

1. Какие основные элементы есть в Kafka?
2. Как происходит доставка сообщений?
3. Как происходит подключение Kafka в коде?

Тайминг: 3 мин

Пишем в чат

/

Говорим голосом

Тезисы

Подведем итоги

1. Kafka – это распределенный, надежный и производительный брокер сообщений
2. Основные элементы Kafka – **Producer, Consumer, Topic, Partition, Replica**
3. Репликация через Leader и In-Sync Replica партиции
4. Кластеризация с zookeeper или Kraft
5. Тестирование через Embedded Kafka, MockProducer, MockConsumer

Домашнее задание

1. Выбрать фреймворк и реализовать прием и отправку сообщений с его помощью
2. Сделать тесты приложения

Срок: 09.09.2025

Цели вебинара

Проверка достижения целей:

1. Узнаете особенности Kafka
2. Сможете принимать решение о ее применении в проекте
3. Выполните интеграцию бизнес-логики в приложение
4. Узнаете шаблон разработки “Стратегия”

Рефлексия

С какими впечатлениями уходите с вебинара?

Как будете применять на практике то,
что узнали на вебинаре?

Следующий вебинар

05.09.2025

Модуль 5: Фреймворки и асинхронный транспорт

Разработка бессерверного приложения

- Ссылка на вебинар будет в ЛК за 15 минут
- Материалы к занятию в ЛК — можно изучать
- Обязательный материал обозначен красной лентой

Список материалов для изучения

1. https://github.com/otuskotlin/202402-ok-marketplace
2. https://habr.com/ru/post/354486/
3. https://www.baeldung.com/spring-kafka
4. https://dzone.com/articles/kafka-topic-architecture-replication-failover-and
5. https://spring.io/projects/spring-kafka/#overview
6. https://docs.spring.io/spring-kafka/reference/html/
7. https://github.com/gAmUssA/ktor-kafka
8. https://docs.confluent.io/platform/current/tutorials/examples/clients/docs/kotlin.html
9. https://docs.confluent.io/5.5.0/getting-started.html
10. https://kafka.apache.org/documentation/streams/
11. https://camel.apache.org/components/latest/kafka-component.html
12. Kafka Kraft: https://habr.com/ru/companies/slurm/articles/685694/
13. Семантики: https://habr.com/ru/companies/badoo/articles/333046/

Заполните, пожалуйста, опрос о занятии по ссылке в чате

Спасибо за внимание!
Приходите на следующие вебинары